ME5411 Robot Vision and AI

Group Project

Microchip Character Recognition

Group 6

Member 1 Student Number
Member 2 Student Number
Member 3 Student Number
Member 4 Student Number

National University of Singapore
November 2026

Abstract

This project investigates image processing and machine learning techniques for recognizing characters printed on a microchip. A complete recognition procedure is developed, covering image enhancement, spatial filtering, region of interest extraction, binarization, character outline detection, segmentation, and classification. To reduce interference from the textured background, multi-scale Gaussian smoothing and local background subtraction are applied before thresholding. Connected-component filtering and vertical foreground projection are then used to separate touching characters. The resulting procedure successfully extracts all ten characters in the target sequence `HD44780A00`.

For character classification, two approaches are implemented and evaluated using a dataset of 7,112 images belonging to seven character categories. The first approach employs a convolutional neural network (CNN) implemented using NumPy, while the second combines principal component analysis (PCA) with a support vector machine (SVM). Both methods are evaluated using the same held-out test set of 1,778 images. The CNN achieves a classification accuracy of 99.21%, compared with 99.10% for PCA–SVM. When applied to the segmented microchip characters, the CNN correctly recognizes all ten characters, whereas PCA–SVM correctly identifies nine. Although the CNN provides slightly higher recognition accuracy, PCA–SVM requires substantially less recorded fitting time and achieves lower inference latency.

The results demonstrate that both classification methods are effective for the supplied dataset. They also highlight the importance of preprocessing and segmentation quality in the overall recognition procedure, as well as the trade-off between classification performance and computational efficiency.

Contents

Abstract **1**

Introduction **3**

1 Task 1 **5**
- 1.1 Introduction . . . 5
- 1.2 Solution and Results . . . 5
- 1.3 Conclusion . . . 8

2 Task 2 **8**
- 2.1 Introduction . . . 8
- 2.2 Solution and Results . . . 8
- 2.3 Conclusion . . . 11

3 Task 3 **12**
- 3.1 Introduction . . . 12
- 3.2 Solution and Results . . . 12
- 3.3 Conclusion . . . 14

4 Task 4 **15**
- 4.1 Introduction . . . 15
- 4.2 Solution and Results . . . 15
- 4.3 Conclusion . . . 18

5 Task 5 **19**
- 5.1 Introduction . . . 19
- 5.2 Solution and Results . . . 19

5.3 Conclusion 21

6 Task 6 **22**
6.1 Introduction 22
6.2 Solution and Results 22
6.3 Conclusion 25

7 Task 7 **25**
7.1 CNN-based Character Classification 25
7.2 Non-CNN-based Character Classification 32
7.3 Comparison of CNN and Non-CNN Methods 34

Overall Discussion and Conclusion **37**

Implementation and Reproduction Guide **38**

AI Declaration **40**

Introduction

Background

Character recognition is an important application of computer vision and pattern recognition, with applications in industrial inspection, document processing, and automated identification. In practical situations, recognition is often affected by complex background textures, uneven illumination, low contrast, and irregular character boundaries. Therefore, image preprocessing and segmentation are essential for extracting reliable character information before classification [1].

In this project, we investigate the recognition of characters printed on a microchip. The provided image contains the target sequence `HD44780A00`, together with an unrelated upper line of text and substantial background texture. A complete recognition procedure is developed by combining traditional image processing techniques with machine learning classification methods.

Project Objectives

The objective of this project is to extract and recognize the ten target characters from the microchip image and evaluate the effectiveness of different processing and classification methods.

The project consists of two main parts. The first part investigates image enhancement, spatial filtering, region extraction, binarization, contour detection, and character segmentation. The second part develops a convolutional neural network (CNN) and a non-CNN classifier using the provided character dataset, followed by performance evaluation and comparison.

Overview of the Proposed Method

The proposed recognition system consists of two main stages: image processing and character segmentation, followed by machine learning classification.

Image Processing and Character Segmentation

The image processing procedure is summarized in Figure 1.

The original microchip image is converted into grayscale, and contrast enhancement and averaging filters are investigated. A rectangular region of interest (ROI) is then extracted to isolate the target character sequence.

The grayscale ROI is processed using Gaussian smoothing at two different scales. Subtracting the large-scale background estimate from the locally smoothed image produces a contrast response, which is thresholded to obtain a binary foreground mask.

Connected-component filtering removes isolated foreground regions. Character outlines are extracted using four-neighborhood boundary detection, while connected-component geometry and vertical projection are used to separate touching characters.

The resulting ten character regions are labelled and saved for subsequent classification.

Character Classification and Evaluation

The classification procedure is summarized in Figure 2.

The provided dataset contains seven character categories: 0, 4, 7, 8, A, D, and H. Exact duplicate images are grouped before the dataset is divided into training, validation, and test subsets.

Two classification methods are investigated. The CNN is implemented using NumPy and learns spatial features through convolutional and fully connected layers. The non-CNN method uses principal component analysis (PCA) for dimensionality reduction, followed by an RBF-kernel support vector machine (SVM).

Both methods use the same dataset partition. Model configurations are evaluated on the validation subset, and the selected models are assessed using the held-out test set. The trained

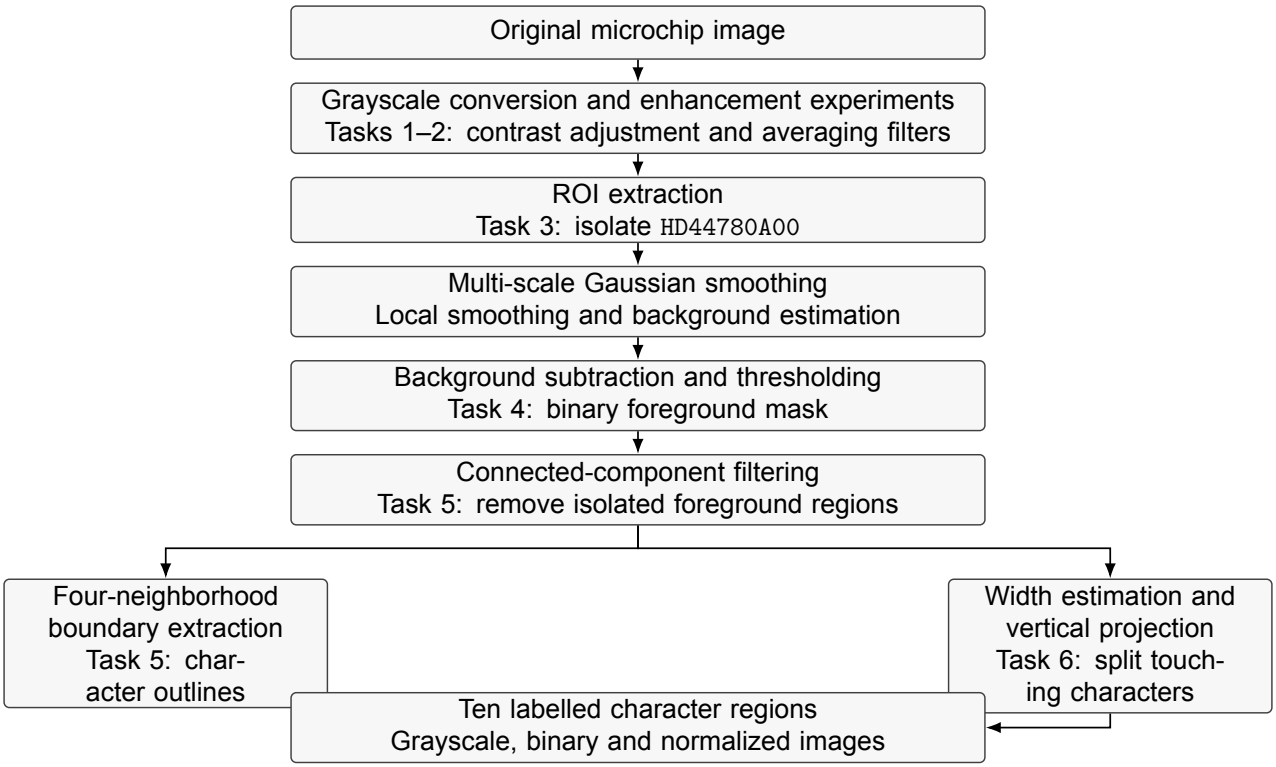


Figure 1: Flowchart of the image processing and character segmentation procedure.

classifiers are subsequently applied to the ten characters extracted from the microchip image.

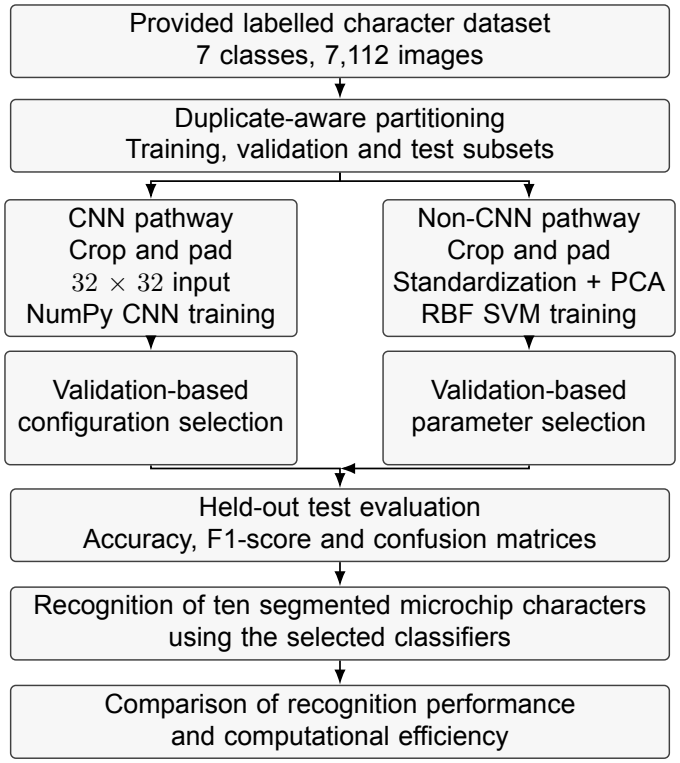


Figure 2: Flowchart of CNN and PCA–SVM training, evaluation, and microchip character recognition.

The two approaches are compared in terms of classification accuracy, class-wise performance, computational efficiency, and recognition results on the provided microchip image.

Report Organization

Tasks 1–3 investigate image enhancement, spatial filtering, and ROI extraction. Tasks 4–6 describe binarization, contour detection, and character segmentation. Task 7 presents the CNN and PCA–SVM classification methods, including their experimental results and performance comparison. The report concludes with an overall discussion and a guide for reproducing the implementation.

1 Task 1

1.1 Introduction

In this task, we focus on loading and inspecting the given microchip image, converting it into grayscale, and applying contrast enhancement to improve the visibility of the characters. The original image contains considerable background texture and relatively weak character boundaries, which may affect subsequent image processing operations.

We first examine the image format and dimensions. The RGB image is then converted into grayscale, followed by automatic contrast stretching. To evaluate the enhancement results, we compare the grayscale histograms before and after processing and calculate the RMS contrast and mean intensity. Finally, we discuss the effectiveness and limitations of global contrast enhancement in the overall image processing procedure.

1.2 Solution and Results

1.2.1 Image Loading and Format Inspection

We begin by reading the original image, `Test_Image_2026.bmp`, using the Python Pillow library. The image is converted into RGB format to ensure a consistent representation for subsequent processing. Its dimensions are inspected before applying any image processing operations.

The original image has a resolution of 990×367 pixels and contains three color channels. As shown in Figure 3, the image contains two lines of characters printed on the surface of a microchip. The upper line is not the target of the recognition task, while the lower line contains the ten characters to be extracted and classified in the following tasks.



Figure 3: Original RGB image of the microchip.

1.2.2 Grayscale Conversion and Contrast Enhancement

Before applying contrast enhancement, the RGB image is converted into grayscale [1].In grayscale representation, each pixel is described by a single intensity value instead of three color components. This reduces the dimensionality of the image and allows subsequent processing to focus on intensity variations rather than color information.

For an RGB image, a commonly used luminance approximation is

$$Y = 0.299R + 0.587G + 0.114B \tag{1}$$

where R, G, and B represent the three color components, and Y denotes the resulting grayscale intensity.

In our implementation, grayscale conversion is performed using the `convert('L')` function provided by Pillow.

After grayscale conversion, automatic contrast stretching is applied using the `ImageOps.autocontrast()` function. The basic idea of contrast stretching is to redistribute the grayscale intensities over a wider range, thereby increasing the intensity differences between relatively bright and dark regions.

For a grayscale image with minimum intensity I_{min} and maximum intensity I_{max}, the linear contrast stretching operation can be expressed as

$$I_{\text{out}}(x, y) = \frac{I_{\text{in}}(x, y) - I_{\text{min}}}{I_{\text{max}} - I_{\text{min}}} \times 255 \tag{2}$$

where $I_{\text{in}}(x, y)$ and $I_{\text{out}}(x, y)$ denote the input and output pixel intensities, respectively. This expression assumes that $I_{\text{max}} > I_{\text{min}}$. In the implementation, Pillow handles the intensity mapping internally.

The corresponding Python implementation is shown below.

```
source = Image.open(args.input).convert('RGB')
gray = source.convert('L')
ImageOps.autocontrast(gray).save(
    out / '02_contrast.png'
)
```

The contrast-enhanced grayscale image is presented in Figure 4.



Figure 4: Grayscale image after automatic contrast stretching.

Comparing Figure 3 and Figure 4, the contrast enhancement changes the intensity distribution while preserving the spatial arrangement of the characters. The enhanced image appears darker overall, and the intensity differences between image regions become more pronounced.

However, the textured background remains visible after enhancement. This suggests that global contrast stretching alone is insufficient to completely separate the characters from the background.

1.2.3 Histogram Analysis

To further examine the effect of contrast enhancement, we compare the grayscale intensity distributions before and after processing.

A grayscale histogram represents the number of pixels corresponding to each intensity level. For an 8-bit grayscale image, the intensity values range from 0 to 255. By comparing the histograms, we can observe how the enhancement operation redistributes the pixel intensities.

The histograms of the original grayscale image and the contrast-enhanced image are shown in Figure 5.

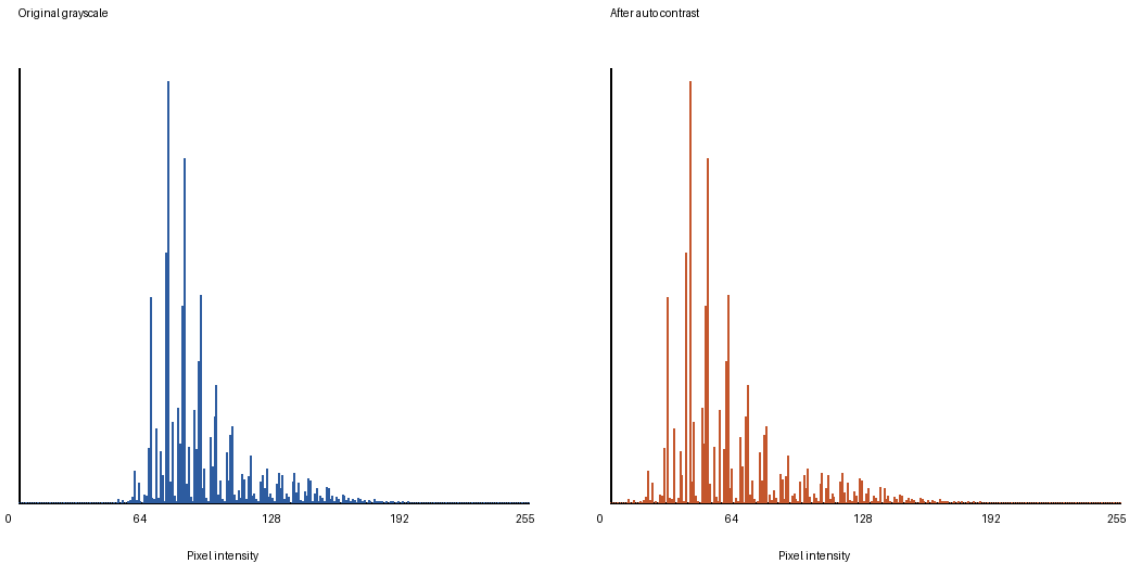


Figure 5: Grayscale intensity histograms before and after automatic contrast enhancement.

As shown in Figure 5, most pixel intensities in the original grayscale image are concentrated within a relatively narrow range. Several prominent peaks can be observed, indicating that a large proportion of pixels have similar intensity values.

After automatic contrast enhancement, the histogram is redistributed over a wider intensity range. The distribution shifts towards lower intensity values, while the relative structure of the major peaks is generally preserved.

This indicates that the contrast stretching operation changes the overall brightness and expands the intensity differences between image regions. However, the enhancement does not eliminate the background texture, which still needs to be addressed in the following image processing tasks.

1.2.4 Contrast and Intensity Measurements

To quantitatively evaluate the enhancement results, we calculate the RMS contrast, which is represented by the standard deviation of grayscale intensities.

For an image containing N pixels, the RMS contrast is defined as

$$C_{\text{RMS}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(I_i - \mu)^2} \tag{3}$$

where I_i represents the grayscale intensity of the i-th pixel and μ is the mean grayscale intensity, calculated by

$$\mu = \frac{1}{N}\sum_{i=1}^{N} I_i \tag{4}$$

In addition to RMS contrast, we also calculate the mean intensity and record the minimum and maximum grayscale values. These measurements allow us to evaluate both the overall intensity variation and the effective grayscale range.

The experimental results are summarized in Table 1.

From Table 1, the RMS contrast increases from 23.60 to 28.64, corresponding to a relative

Table 1: Comparison of grayscale intensity statistics before and after contrast enhancement.

Method	RMS Contrast	Mean	Min	Max
Original grayscale	23.60	91.69	42	252
Auto contrast	28.64	59.86	0	255

improvement of approximately 21.34%.

Meanwhile, the mean grayscale intensity decreases from 91.69 to 59.86, indicating that the enhanced image becomes darker overall. The minimum intensity decreases from 42 to 0, while the maximum intensity increases from 252 to 255, showing that the available grayscale range is more fully utilized.

These results demonstrate that automatic contrast stretching effectively increases global intensity variation. However, the enhancement also affects the background texture. Therefore, a higher global contrast value does not necessarily indicate better character segmentation performance.

In the subsequent segmentation pipeline, the original grayscale region of interest is used as the input for Gaussian smoothing and local background subtraction, rather than directly using the globally contrast-enhanced image. This allows the segmentation procedure to address local background variations separately.

1.3 Conclusion

In this task, the original microchip image was successfully loaded and converted into grayscale. Automatic contrast enhancement was implemented using Pillow, and its effectiveness was evaluated through visual comparison, histogram analysis, and quantitative intensity measurements.

The experimental results show that the RMS contrast increases by approximately 21.34%, while the grayscale range expands from 42–252 to 0–255. The enhancement increases global intensity variation but does not eliminate the textured background.

Therefore, although automatic contrast stretching is effective in improving global contrast, additional preprocessing is still required for reliable character extraction. The following tasks investigate image smoothing, region extraction, binarization, and segmentation to further process the microchip image.

2 Task 2

2.1 Introduction

In this task, spatial averaging filters are applied to the microchip image to investigate the effects of smoothing on background texture and character visibility. The required 5×5 averaging filter is evaluated together with 3×3 and 7×7 filters to examine the influence of neighborhood size.

The results are compared through visual inspection and quantitative measurements. Gaussian smoothing, which is used in the subsequent character extraction procedure, is also introduced to explain the choice of smoothing strategy.

2.2 Solution and Results

2.2.1 Theoretical Background

Averaging Filter An averaging filter replaces each pixel intensity with the arithmetic mean of the intensities within a local neighborhood [1]. For a square filter of size $k \times k$, the output image $g(x, y)$ is defined as

$$g(x,y) = \frac{1}{k^2} \sum_{i=-r}^{r} \sum_{j=-r}^{r} f(x+i, y+j), \qquad r = \frac{k-1}{2}, \tag{5}$$

where $f(x, y)$ is the input grayscale image and k is an odd positive integer.
The corresponding 5×5 averaging kernel is

$$H_{5\times5} = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}. \tag{6}$$

All kernel elements have equal weights, and their sum is one. The filter reduces local intensity fluctuations, but increasing the neighborhood size may also blur narrow character strokes and boundaries.

Gaussian Smoothing Gaussian smoothing uses spatially varying weights, assigning greater importance to pixels closer to the center of the neighborhood. The two-dimensional Gaussian function is

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \tag{7}$$

where σ controls the smoothing scale. The filtered image is obtained by convolution:

$$g(x, y) = G_\sigma * f(x, y). \tag{8}$$

A smaller σ retains more local detail, while a larger value produces stronger smoothing. In the final character extraction pipeline, Gaussian smoothing is used together with local background subtraction, as described in Task 4.

2.2.2 Implementation

The averaging filter is implemented using NumPy. The grayscale image is converted into a floating-point array, and its boundaries are extended by replicating the nearest edge pixels. The values within each neighborhood are accumulated and divided by the number of kernel elements.
The core implementation is shown below.

```
def mean_filter(image, size):
    a = np.asarray(image, dtype=float)
    p = np.pad(a, size//2, mode='edge')
    total = np.zeros_like(a)

    for y in range(size):
        for x in range(size):
            total += p[
                y:y+a.shape[0],
                x:x+a.shape[1]
            ]

    return Image.fromarray(
        np.uint8(
            np.round(total / (size * size))
        )
    )
```

Three averaging filters are applied to the same original grayscale image:

$$k \in \{3, 5, 7\}. \tag{9}$$

The input image and processing procedure remain unchanged across the experiments, allowing the influence of filter size to be examined independently.

2.2.3 Experimental Results

The filtered images obtained using the three neighborhood sizes are shown in Figures 6, 7, and 8.



Figure 6: Averaging filter result with a 3×3 neighborhood.

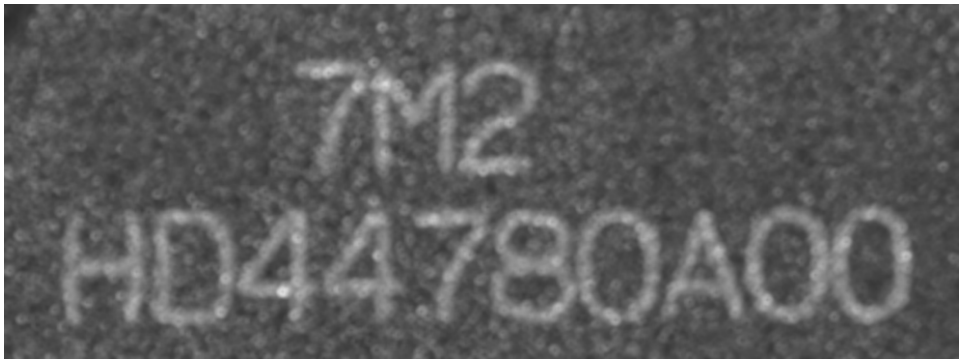


Figure 7: Averaging filter result with a 5×5 neighborhood.

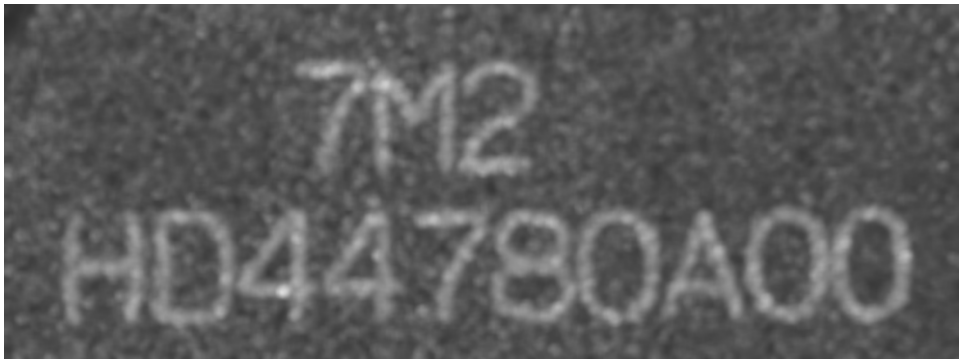


Figure 8: Averaging filter result with a 7×7 neighborhood.

The 3×3 filter preserves relatively clear character boundaries but leaves noticeable granular background texture. The 5×5 filter produces stronger smoothing while retaining the main character shapes. With the 7×7 filter, the background becomes smoother, but character edges and narrow gaps appear less distinct.

These results demonstrate the trade-off between noise suppression and preservation of spatial details.

2.2.4 Quantitative Comparison

To quantify the smoothing effects, the RMS contrast and mean grayscale intensity are calculated for each filtered image. The RMS contrast is defined in Task 1 as the population standard deviation of pixel intensities.

The results are summarized in Table 2.

Table 2: Quantitative comparison of averaging filters.

Filter Size	RMS Contrast	Mean Intensity
3×3	22.65	91.69
5×5	21.79	91.69
7×7	21.05	91.69

The RMS contrast decreases from 22.65 to 21.05 as the filter size increases, while the mean intensity remains approximately 91.69. This indicates that larger averaging windows reduce intensity variations without substantially changing the overall image brightness.

Table 3 summarizes the characteristics of the three filters.

Table 3: Characteristics of different averaging filter sizes.

Property	3×3	5×5	7×7
Pixels per window	9	25	49
Kernel weight	$1/9$	$1/25$	$1/49$
Smoothing strength	Mild	Moderate	Strong
Detail preservation	Higher	Moderate	Lower

Among the tested filters, the 5×5 averaging filter provides a reasonable visual compromise between smoothing strength and character detail preservation. However, this observation does not establish that it produces the highest downstream segmentation accuracy.

2.2.5 Discussion

The averaging filter experiments show that increasing neighborhood size suppresses local intensity fluctuations but also reduces the sharpness of character boundaries. Although the 5×5 filter offers a balanced result, the filtered image still contains substantial background texture.

The final segmentation pipeline therefore uses Gaussian smoothing and local background subtraction rather than directly using the mean-filtered images. Gaussian smoothing assigns larger weights to nearby pixels, while the background subtraction stage reduces slowly varying intensity components. The combined procedure is examined in Task 4.

2.3 Conclusion

The required 5×5 averaging filter was implemented using NumPy and compared with 3×3 and 7×7 filters. Both visual inspection and RMS contrast measurements confirmed that larger neighborhoods produce stronger smoothing at the expense of character boundary sharpness.

The 5×5 filter provided a reasonable balance among the tested configurations. The results also demonstrated that smoothing alone is insufficient for isolating the characters from the textured background, motivating the local contrast-based processing used in subsequent tasks.

3 Task 3

3.1 Introduction

After investigating contrast enhancement and image smoothing in the previous tasks, we proceed to extract the region containing the target characters. The original microchip image contains two lines of text as well as a considerable amount of background texture. However, only the lower line, `HD44780A00`, is required for subsequent character segmentation and recognition.

Processing the entire image would introduce unnecessary background information and characters that are not relevant to the recognition task. Therefore, a region of interest (ROI) is selected to isolate the target character line.

The objective of this task is to extract a sub-image containing all ten target characters while excluding most of the unrelated image regions. The extracted ROI will serve as the input for subsequent smoothing, local background estimation, binarization, and character segmentation.

3.2 Solution and Results

3.2.1 Theoretical Background

Region of interest extraction is a fundamental operation in image processing. It selects a specific spatial region from an image so that subsequent operations can focus on the relevant information.

For a rectangular ROI, the selected region can be represented by four coordinates:

$$R = (x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}}) \tag{10}$$

where $(x_{\text{min}}, y_{\text{min}})$ represents the upper-left corner and $(x_{\text{max}}, y_{\text{max}})$ represents the lower-right boundary of the selected region.

The width and height of the extracted ROI are calculated as

$$W_{\text{ROI}} = x_{\text{max}} - x_{\text{min}} \tag{11}$$

$$H_{\text{ROI}} = y_{\text{max}} - y_{\text{min}} \tag{12}$$

For the present image, the target characters are arranged approximately along a horizontal line. Therefore, a rectangular ROI is suitable for isolating the complete character sequence.

Restricting the processing area provides several advantages. First, it excludes the upper line of characters and most unrelated background regions. Second, it reduces the number of pixels processed in subsequent operations. Third, it allows the following binarization and segmentation procedures to focus on the target character line.

However, the ROI must be selected carefully. An excessively small region may truncate character strokes, while an unnecessarily large region may introduce additional background noise and interfere with subsequent processing.

3.2.2 Implementation

In our implementation, the ROI is extracted from the original grayscale image using the `crop()` function provided by the Python Pillow library.

Unlike interactive cropping, which requires the user to manually select a region each time, we specify the ROI using fixed pixel coordinates. This allows the same region to be extracted consistently when the program is executed repeatedly.

The selected ROI coordinates are

$$R = (10, 185, 980, 350) \tag{13}$$

The resulting dimensions are

$$W_{\text{ROI}} = 980 - 10 = 970 \tag{14}$$

$$H_{\text{ROI}} = 350 - 185 = 165 \tag{15}$$

Therefore, the extracted sub-image has a resolution of 970×165 pixels.

The main Python implementation is shown below.

```
roi = tuple(args.roi)

crop = gray.crop(roi)

crop.save(out / '04_roi.png')
```

The default ROI coordinates are defined in the program as follows.

```
parser.add_argument(
    '--roi',
    nargs=4,
    type=int,
    default=[10, 185, 980, 350],
    metavar=('X0', 'Y0', 'X1', 'Y1')
)
```

The program also checks whether the selected ROI lies within the original image boundaries and whether the specified region has positive width and height. This prevents invalid cropping coordinates from being used.

Although the default coordinates are fixed, the ROI can still be adjusted through the command-line argument when processing images with different target positions.

3.2.3 Experimental Results

The original image is shown in Figure 3. It contains the target character sequence together with the upper line of text and surrounding background texture.

After ROI extraction, the resulting sub-image is shown in Figure 9.

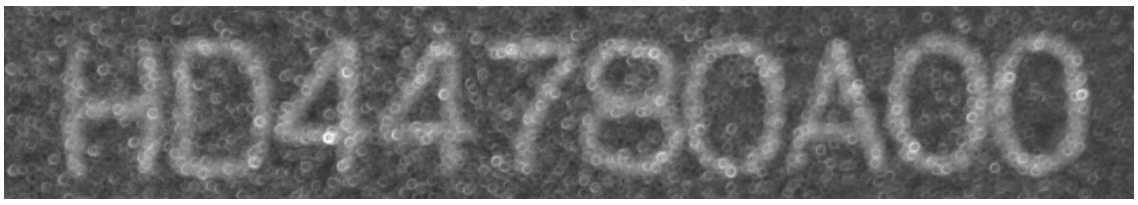


Figure 9: Extracted region of interest containing the target character sequence HD44780A00.

As shown in Figure 9, the extracted ROI contains the complete target character sequence `HD44780A00`. The upper line of characters has been excluded, while the main structures of the ten target characters remain within the selected region.

The cropping operation also removes most of the background areas above and below the target line. However, the textured background within the ROI remains visible, since cropping only selects a spatial region and does not directly suppress image noise.

The extracted image is therefore suitable for further preprocessing, but additional operations are still necessary to distinguish the character strokes from the background.

3.2.4 Discussion

The ROI extraction method used in this project is based on fixed spatial coordinates. For the provided microchip image, this approach is simple and reproducible because the target character line occupies a known position.

Compared with processing the complete image, ROI extraction reduces the image area from 990×367 pixels to 970×165 pixels.

The original image contains

$$N_{\text{original}} = 990 \times 367 = 363330 \tag{16}$$

pixels, while the extracted ROI contains

$$N_{\text{ROI}} = 970 \times 165 = 160050 \tag{17}$$

pixels.

Therefore, the number of pixels processed in the following stages is reduced by approximately

$$\frac{363330 - 160050}{363330} \times 100\% \approx 55.95\% \tag{18}$$

This reduction decreases the spatial extent of subsequent image processing operations and eliminates the need to process the unrelated upper line of characters.

Nevertheless, the fixed-coordinate method has a limitation. If the position or scale of the target characters changes significantly, the predefined ROI may no longer contain the complete character sequence. In such cases, the ROI coordinates would need to be adjusted or an automatic localization method would be required.

For the given image, the fixed ROI provides a straightforward and consistent solution without introducing additional localization algorithms.

It should also be noted that the ROI is extracted directly from the original grayscale image rather than from the contrast-enhanced or mean-filtered images generated in Tasks 1 and 2. In the subsequent processing pipeline, Gaussian smoothing and local background estimation are applied to this grayscale ROI before binarization.

This design allows the preprocessing operations used for character segmentation to be performed directly on the selected target region.

3.3 Conclusion

In this task, a rectangular region of interest was successfully extracted from the original grayscale microchip image using Pillow.

The selected ROI, defined by the coordinates $(10, 185, 980, 350)$, produces a sub-image of size 970×165 pixels containing the complete character sequence `HD44780A00`.

Compared with the original image, the ROI reduces the number of pixels by approximately 55.95% and excludes most unrelated image regions. The fixed-coordinate implementation also provides consistent and reproducible extraction for the given input image.

However, the extracted ROI still contains substantial background texture. Therefore, further preprocessing is required before the individual characters can be reliably separated.

The following tasks investigate local intensity processing, image binarization, character outline detection, and segmentation.

4 Task 4

4.1 Introduction

In this task, the grayscale region of interest (ROI) obtained in Task 3 is converted into a binary image to separate the character strokes from the textured background. Since the original image contains non-uniform intensity variations, direct thresholding of the grayscale ROI may retain unwanted background regions or remove weak character details.

To address this problem, we use multi-scale Gaussian smoothing and local background subtraction before thresholding. The intermediate processing results are examined, and a threshold sensitivity experiment is conducted to evaluate the balance between background suppression and character preservation.

4.2 Solution and Results

4.2.1 Theoretical Background

Image Binarization Binarization assigns each pixel to either the foreground or background [1]. For a grayscale image $I(x, y)$ and threshold T, the basic operation is defined as

$$B(x,y) = \begin{cases} 1, & I(x,y) > T, \\ 0, & I(x,y) \leq T. \end{cases} \tag{19}$$

Here, foreground pixels are represented by 1 and background pixels by 0. However, a single threshold applied directly to the original grayscale image may be unreliable when foreground and background intensities overlap. Therefore, a local contrast response is calculated before binarization.

Multi-scale Gaussian Smoothing Gaussian smoothing replaces each pixel with a weighted average of its neighboring intensities. The two-dimensional Gaussian function is

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right), \tag{20}$$

where σ controls the smoothing scale.

Two Gaussian filters are applied to the grayscale ROI:

$$I_s = G_{\sigma_s} * I, \qquad I_b = G_{\sigma_b} * I, \tag{21}$$

where I_s is the locally smoothed image and I_b is the large-scale background estimate. The selected parameters are

$$\sigma_s = 3, \qquad \sigma_b = 25. \tag{22}$$

The smaller smoothing scale reduces fine intensity fluctuations while retaining the main character structures. The larger scale suppresses local details and represents slowly varying background intensity.

Local Background Subtraction The local contrast response is calculated by subtracting the background estimate from the locally smoothed image:

$$R(x,y) = I_s(x,y) - I_b(x,y). \tag{23}$$

This difference-of-Gaussians operation emphasizes structures that are brighter than their local surroundings while reducing broad background variations.

For visualization, the response is shifted and clipped to the 8-bit grayscale range:

$$V(x, y) = \text{clip}\left(R(x, y) + 128, 0, 255\right). \tag{24}$$

The shifted response is used only for visualization. The binary threshold is applied directly to the unshifted response $R(x, y)$.

4.2.2 Implementation

The binarization procedure is implemented using Pillow and NumPy. Two Gaussian filters with standard deviations of 3 and 25 are applied to the grayscale ROI. Their difference is calculated using floating-point arrays to avoid unsigned integer subtraction errors.

The main implementation is shown below.

```
smoothed = crop.filter(
    ImageFilter.GaussianBlur(args.blur)
)

background = crop.filter(
    ImageFilter.GaussianBlur(args.background)
)

response = (
    np.asarray(smoothed, dtype=float)
    - np.asarray(background, dtype=float)
)

binary = response > args.threshold
```

The default values are $\sigma_s = 3$, $\sigma_b = 25$, and $T = 10$. Therefore, the binary mask is obtained using

$$B(x, y) = \begin{cases} 1, & R(x, y) > 10, \\ 0, & R(x, y) \leq 10. \end{cases} \tag{25}$$

The binary array is converted into an 8-bit image, where foreground and background pixels are displayed as white and black, respectively.

4.2.3 Experimental Results

The intermediate results of the binarization procedure are presented in Figures 10–13.

Gaussian Smoothing and Background Estimation Figure 10 shows the ROI after Gaussian smoothing with $\sigma_s = 3$. The main character structures remain visible, although some background texture is still present.



Figure 10: Gaussian-smoothed ROI with $\sigma_s = 3$.

Figure 11 shows the background estimate obtained using $\sigma_b = 25$. Most local character details are blurred, leaving broad intensity variations across the ROI.



Figure 11: Large-scale Gaussian background estimate with $\sigma_b = 25$.

Local Contrast Response The difference between the two smoothed images is shown in Figure 12. Subtracting the background estimate reduces broad intensity variations and makes the bright character structures more distinguishable. However, some unwanted local structures still produce positive responses.



Figure 12: Local contrast response obtained by Gaussian background subtraction.

Binary Image The binary image obtained using $T = 10$ is shown in Figure 13.



Figure 13: Binary image obtained using the threshold $T = 10$.

The main structures of all ten characters are retained, while much of the background is suppressed. Some isolated foreground pixels and connections between neighboring characters remain, requiring further processing in Tasks 5 and 6.

4.2.4 Sensitivity Analysis of the Binarization Threshold

To examine the influence of threshold selection, four values were tested:

$$T \in \{5, 10, 15, 20\}. \tag{26}$$

The Gaussian smoothing parameters were kept fixed at $\sigma_s = 3$ and $\sigma_b = 25$ so that only the threshold varied.

For each threshold, the foreground pixel ratio and the number of four-connected components were calculated. The foreground pixel ratio is defined as

$$P_{\text{fg}} = \frac{N_{\text{fg}}}{N_{\text{total}}} \times 100\%, \tag{27}$$

where N_{fg} is the number of foreground pixels and N_{total} is the total number of pixels in the ROI.

The resulting binary images are compared in Figure 14.

The quantitative results are summarized in Table 4.

At $T = 5$, the character strokes appear relatively thick, but numerous small foreground regions remain in the background. Increasing the threshold to $T = 10$ reduces the foreground ratio from 31.02% to 26.21% and the number of connected components from 58 to 34, while retaining the main character structures.

At $T = 15$, the foreground ratio decreases to 22.31%, and the background becomes cleaner. Further increasing the threshold to $T = 20$ reduces the foreground ratio to 18.53%, but some narrow stroke regions become less continuous.

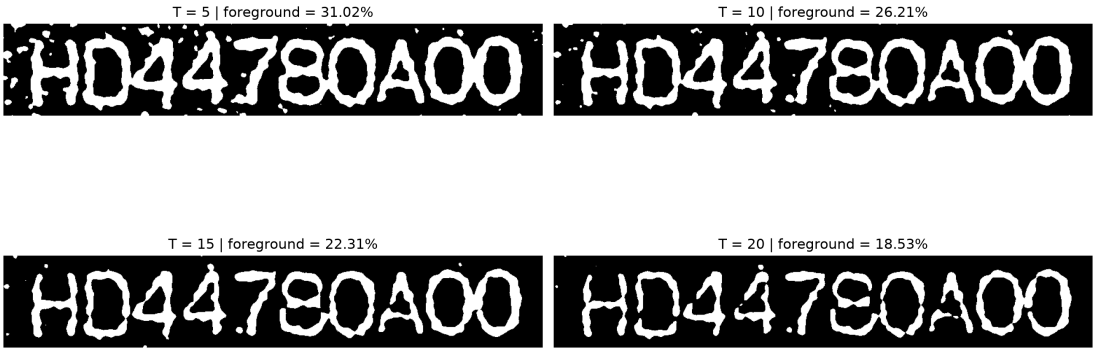


Figure 14: Binary images obtained using four threshold values with fixed Gaussian smoothing parameters.

Table 4: Sensitivity of the binarization results to threshold selection.

Threshold	Foreground (%)	Components	Components (≥ 10 pixels)
5	31.02	58	47
10	26.21	34	27
15	22.31	19	16
20	18.53	19	17

The connected-component count decreases from 58 at $T = 5$ to 19 at $T = 15$ and remains at 19 for $T = 20$. However, this count alone does not determine segmentation quality, since a component may represent noise, a fragmented stroke, or multiple touching characters.

Among the tested values, $T = 10$ provides a reasonable balance between suppressing background noise and preserving character structures. It is therefore retained for subsequent processing.

4.2.5 Discussion

The experimental results demonstrate that local background subtraction is effective for emphasizing bright character structures against the textured microchip background. The smaller Gaussian scale preserves local details, while the larger scale estimates slowly varying background intensity.

The threshold sensitivity experiment further shows that stronger background suppression is accompanied by a reduction in foreground pixels, which may weaken character strokes. The selected threshold retains the main structures of the ten characters while removing much of the unwanted background.

However, the resulting binary image still contains small isolated components and touching character regions. These are addressed through connected-component filtering and geometric segmentation in the subsequent tasks. Therefore, the binary image serves as an intermediate representation rather than the final character segmentation result.

4.3 Conclusion

The grayscale ROI was successfully converted into a binary image using multi-scale Gaussian smoothing, local background subtraction, and thresholding. Gaussian smoothing scales of 3 and 25 were used, followed by a threshold of $T = 10$.

The sensitivity experiment showed that increasing the threshold from 5 to 20 reduced the foreground pixel ratio from 31.02% to 18.53% and the number of four-connected components from 58 to 19. Based on the observed trade-off between noise suppression and stroke preservation, $T = 10$ was retained for subsequent character extraction.

5 Task 5

5.1 Introduction

In this task, the outlines of the characters are extracted from the binary image obtained in Task 4. Before boundary extraction, connected-component filtering is applied to remove isolated foreground regions while retaining the main character structures. The cleaned image is then processed using a four-neighborhood boundary detection method. The results are evaluated through visual inspection and quantitative measurements.

5.2 Solution and Results

5.2.1 Theoretical Background

Connected-component Filtering A connected component is a group of foreground pixels connected according to a specified neighborhood relationship. Four-connectivity is used in this project, where pixels are connected through their horizontal or vertical neighbors.

For a pixel at (x, y), its four-neighborhood is defined as

$$N_4(x, y) = \{(x-1, y), (x+1, y), (x, y-1), (x, y+1)\}. \tag{28}$$

To remove unwanted foreground components, each connected region C_i is examined according to its area and vertical extent:

$$A_i = |C_i|, \qquad h_i = y_{\max,i} - y_{\min,i}. \tag{29}$$

A component is retained when

$$A_i \geq A_{\min}, \qquad h_i \geq 0.4 H_{\text{ROI}}, \tag{30}$$

where $A_{\min} = 100$ pixels is the minimum component area and H_{ROI} is the ROI height. These conditions suppress small or vertically insignificant regions while retaining the main character structures.

Four-neighborhood Boundary Extraction A foreground pixel is considered an interior pixel when all four of its direct neighbors also belong to the foreground [1].

For a binary mask $B(x, y)$, the interior mask is defined as

$$\begin{aligned} I(x, y) = B(x, y) \wedge B(x-1, y) \wedge B(x+1, y) \\ \wedge B(x, y-1) \wedge B(x, y+1), \end{aligned} \tag{31}$$

where $\wedge$ denotes the logical AND operation.

The boundary mask is obtained by removing interior pixels:

$$E(x, y) = B(x, y) \wedge \neg I(x, y). \tag{32}$$

This operation is equivalent to binary morphological boundary extraction using a cross-shaped structuring element:

$$E = B - (B \ominus S), \tag{33}$$

where $\ominus$ denotes binary erosion and S represents the four-neighborhood structuring element.

The method extracts both external character outlines and the boundaries of enclosed regions, such as those found in 0, 8, and D.

5.2.2 Implementation

The connected components are first examined according to their area and height. Components satisfying the filtering criteria are retained in a cleaned binary mask.

```
cleaned = np.zeros_like(binary)
boxes = []

for box, pts in components(binary):
    if (len(pts) >= args.min_area
        and box[3] - box[1]
        >= 0.4 * crop.height):
        cleaned[pts[:, 0], pts[:, 1]] = True
        boxes.append(box)
```

The cleaned mask is then padded with a background border. Interior pixels are identified using logical operations on the four neighboring positions.

```
padded = np.pad(cleaned, 1)

interior = (
    cleaned
    & padded[:-2, 1:-1]
    & padded[2:, 1:-1]
    & padded[1:-1, :-2]
    & padded[1:-1, 2:]
)

contours = cleaned & ~interior
```

The cleaned binary image and the extracted contours are saved as `09_cleaned.png` and `10_contours.png`, respectively.

5.2.3 Experimental Results

Connected-component Filtering The cleaned binary image is shown in Figure 15.



Figure 15: Binary image after connected-component filtering.

Compared with the binary image obtained in Task 4, the cleaned image contains fewer isolated foreground regions while preserving the main character structures. However, some neighboring characters remain connected and are therefore represented by the same component.

Character Outline Extraction The extracted character outlines are shown in Figure 16.

The contour image retains the boundaries of the foreground structures while excluding their interior pixels. Both external character boundaries and internal hole boundaries are visible. Some irregularities remain where the original binary mask contains rough or connected strokes.



Figure 16: Character outlines extracted using four-neighborhood boundary detection.

5.2.4 Quantitative Evaluation of Component Filtering

To evaluate the cleaning operation, the numbers of foreground pixels and four-connected components are measured before and after filtering. The results are summarized in Table 5.

Table 5: Quantitative evaluation of connected-component filtering.

Measurement	Before Cleaning	After Cleaning
Foreground pixels	41,952	41,104
Four-connected components	34	7

The filtering operation removes 848 foreground pixels, corresponding to 2.02% of the original foreground. Meanwhile, the number of connected components decreases from 34 to 7, a reduction of 79.41%.

This indicates that the operation removes many isolated components while retaining approximately 97.98% of the foreground pixels. The result is consistent with the removal of small background artifacts rather than extensive modification of the character structures.

After boundary extraction, the contour image contains 5,127 pixels, corresponding to approximately 12.47% of the cleaned foreground pixels.

The seven remaining connected components do not correspond directly to seven individual characters, since some neighboring characters remain connected. Their separation is addressed in Task 6.

5.2.5 Discussion

The four-neighborhood method extracts character boundaries directly from the cleaned binary mask using logical array operations. It preserves both external outlines and internal hole boundaries without requiring additional grayscale processing.

The quantitative results show that component filtering substantially reduces the number of isolated foreground regions while preserving most character pixels. However, boundary quality still depends on the binary mask: broken strokes produce discontinuous outlines, while touching characters may form connected boundaries.

Therefore, the extracted contour image is primarily used to visualize character structures. The subsequent segmentation procedure operates on the cleaned binary mask and connected-component bounding boxes to separate touching characters.

5.3 Conclusion

Connected-component filtering and four-neighborhood boundary detection were applied to extract the character outlines. The filtering stage reduced the number of connected components from 34 to 7 while removing only 2.02% of the foreground pixels. The resulting contour image retained the main external and internal character boundaries.

Although the outlines were successfully extracted, some neighboring characters remained connected. Further geometric segmentation is therefore required to obtain ten individual character regions.

6 Task 6

6.1 Introduction

In this task, the cleaned binary image is segmented into individual character regions and labelled from left to right. The target sequence is `HD44780A00`, containing ten characters. However, the cleaned image contains only seven connected components because some neighboring characters remain connected.

To separate these regions, we combine connected-component geometry, character width estimation, and vertical foreground projection. The resulting character images are then normalized and saved for subsequent classification.

6.2 Solution and Results

6.2.1 Theoretical Background

Connected-component Analysis Four-connected components are identified from the cleaned binary mask using the standard concept of pixel connectivity in binary image analysis [1]. For each component C_i, a bounding box is calculated:

$$B_i = (x_{0,i}, y_{0,i}, x_{1,i}, y_{1,i}), \tag{34}$$

with width and height

$$w_i = x_{1,i} - x_{0,i}, \qquad h_i = y_{1,i} - y_{0,i}. \tag{35}$$

A connected component may contain more than one character when adjacent strokes touch. Therefore, the bounding-box width is used to identify regions requiring further splitting.

Character Width Estimation The component widths are sorted in ascending order:

$$w_{(1)} \le w_{(2)} \le \cdots \le w_{(n)}. \tag{36}$$

The typical character width is estimated as the median of the narrower half of the components:

$$w_{\text{typical}} = \text{median}\left(w_{(1)}, \ldots, w_{(m)}\right), \qquad m = \left\lceil \frac{n}{2} \right\rceil. \tag{37}$$

This reduces the influence of unusually wide components containing touching characters. For the present image, the estimated typical width is 81 pixels.

A component is considered for splitting when

$$w_i > 1.45 w_{\text{typical}}. \tag{38}$$

The estimated number of characters in such a component is

$$n_i = \max\left(1, \left\lfloor \frac{w_i}{w_{\text{typical}}} + 0.5 \right\rfloor\right). \tag{39}$$

Components below the width threshold are treated as single characters.

Vertical Foreground Projection For a component mask $M(x, y)$, the vertical foreground projection is defined as

$$P(x) = \sum_{y=y_0}^{y_1-1} M(x, y). \tag{40}$$

The projection counts foreground pixels in each column. High values generally correspond to character strokes, while lower values may indicate gaps between neighboring characters.

To suppress small fluctuations, a five-point moving average is applied:

$$\overline{P}(x) = \frac{1}{5} \sum_{j=-2}^{2} P(x+j). \tag{41}$$

For a component estimated to contain n_i characters, initial splitting positions are determined by dividing its width into n_i sections. Each position is then refined by searching for a minimum of the smoothed projection within

$$x_{\text{target}} \pm 0.25 w_{\text{typical}}. \tag{42}$$

This procedure selects low-density columns near the expected character boundaries rather than relying solely on equal-width division.

6.2.2 Implementation

The segmentation procedure is implemented using NumPy and Pillow. It operates on the cleaned binary mask obtained in Task 5.

First, the typical character width is calculated from the connected-component bounding boxes:

```
widths = sorted(b[2] - b[0] for b in boxes)
typical = float(np.median(
    widths[:max(1, (len(widths)+1)//2)]
))
```

For each component, the number of characters is estimated from its width:

```
count = (
    max(1, int(np.floor(
        width / typical + 0.5
    )))
    if width > 1.45 * typical
    else 1
)
```

For components requiring splitting, the vertical projection is calculated and smoothed:

```
projection = mask[
    y0:y1, x0:x1
].sum(axis=0).astype(float)

smooth = np.convolve(
    projection, np.ones(5)/5,
    mode='same'
)
```

The program searches for the minimum smoothed projection near each expected boundary. If several columns share the minimum value, the one closest to the expected position is selected.

After splitting, the foreground bounding boxes are refined and sorted from left to right. The ROI coordinates are converted to the original image coordinates using

$$x_{\text{original}} = x_{\text{ROI}} + 10, \qquad y_{\text{original}} = y_{\text{ROI}} + 185. \tag{43}$$

The final boxes are drawn on the original image, and each region is assigned a numerical label. The coordinates and segmentation parameters are also saved in `segmentation.json`.

6.2.3 Experimental Results

Character Localization and Labelling The final localization result is shown in Figure 17.

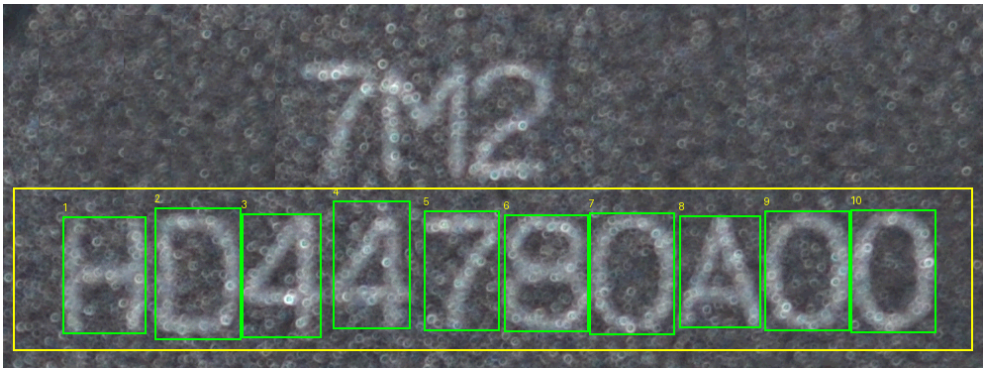


Figure 17: Localization and labelling of the ten target characters in the original microchip image.

The yellow rectangle indicates the ROI, while the green boxes identify the individual character regions. All ten target characters are labelled in their horizontal order, and the unrelated upper line is excluded.

Projection-based Splitting The vertical foreground projection is shown in Figure 18.

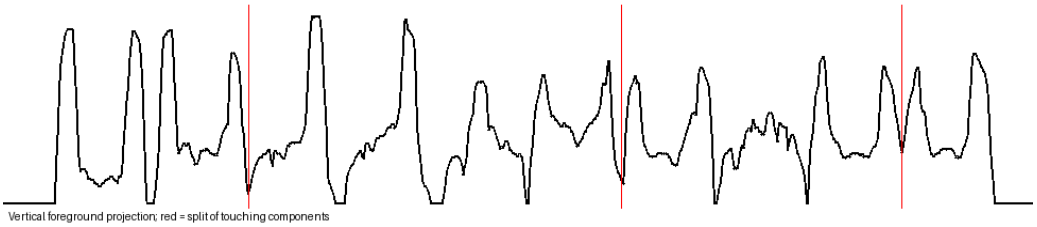


Figure 18: Vertical foreground projection and the selected splitting positions for touching character regions.

The projection curve represents the foreground pixel distribution along the horizontal direction. The red vertical lines indicate the selected splitting positions at ROI coordinates 231, 582, and 846.

These three cuts divide the wider connected regions, increasing the number of character regions from seven to ten.

Extracted Character Regions The ten extracted characters are shown in Figure 19.

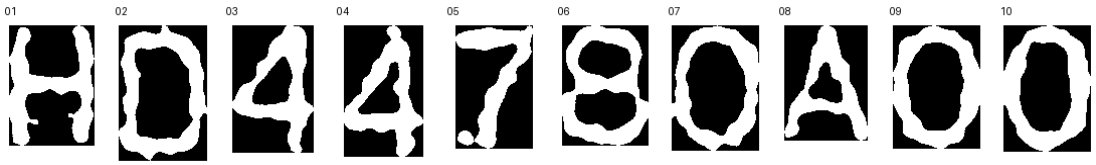


Figure 19: Ten extracted character regions arranged in left-to-right order.

The regions correspond to the sequence `HD44780A00`. Visual inspection confirms that the ten characters have been separated and their main structural features preserved. Some boundary irregularities remain, but the extracted regions provide suitable candidates for classification.

6.2.4 Quantitative Evaluation

The segmentation results are summarized in Table 6.

The initial seven connected components are separated into ten character regions through width estimation and projection-guided splitting. The final bounding boxes were visually checked against the original image, with no apparent missing or additional character regions.

Table 6: Summary of character segmentation results.

Measurement	Result
Connected components after cleaning	7
Typical character width	81 pixels
Projection-based splitting positions	3
Final character regions	10
Expected character regions	10

However, obtaining the expected number of regions does not guarantee correct classification. The extracted character shapes must also be recognized by the classifiers in Task 7.

6.2.5 Character Image Preparation

For each segmented character, the program saves grayscale, binary, and normalized images. The grayscale images are cropped from the original grayscale ROI, while the binary images are obtained from the cleaned foreground mask.

For normalization, the binary character is inverted to obtain a black character on a white background. The character is resized while preserving its aspect ratio and centered on a 64×64 white canvas with a nominal margin of six pixels.

This provides a consistent image representation while avoiding distortion of the original character proportions.

6.2.6 Discussion

The segmentation results demonstrate that connected-component analysis alone is insufficient when neighboring characters touch. Width estimation identifies unusually wide components, while vertical projection provides candidate cutting positions based on foreground density.

Selecting local projection minima near expected boundaries reduces the likelihood of cutting through dense character strokes. For the provided image, the method successfully produces ten character regions from seven connected components.

However, the method assumes approximately horizontal character alignment and comparable character widths. Irregular spacing, substantial overlap, or large differences in character size may affect the estimated number of characters and the selected cutting positions. The fixed ROI also limits direct application to images with different character locations.

6.3 Conclusion

The cleaned binary image was segmented into ten individual character regions using connected-component analysis, width estimation, and vertical foreground projection. A typical character width of 81 pixels was obtained, and three splitting positions were selected to separate touching regions.

The final ten character regions correspond to `HD44780A00`. Their grayscale, binary, and normalized representations were saved for the classification procedures described in Task 7.

7 Task 7

7.1 CNN-based Character Classification

7.1.1 Introduction

A convolutional neural network (CNN) is implemented to classify seven character categories: 0, 4, 7, 8, A, D, and H. CNN-based recognition methods have been extensively investigated for character and document recognition [2]. The network is developed using NumPy, including convolution,

pooling, fully connected layers, backpropagation, and parameter optimization. Its performance is evaluated using a held-out test set, while additional experiments examine preprocessing and network configurations.

7.1.2 Dataset Preparation

Dataset Organization The provided dataset contains 7,112 grayscale images, with 1,016 samples for each of the seven classes. The original image resolution is 128×128 pixels. Representative samples are shown in Figure 20.

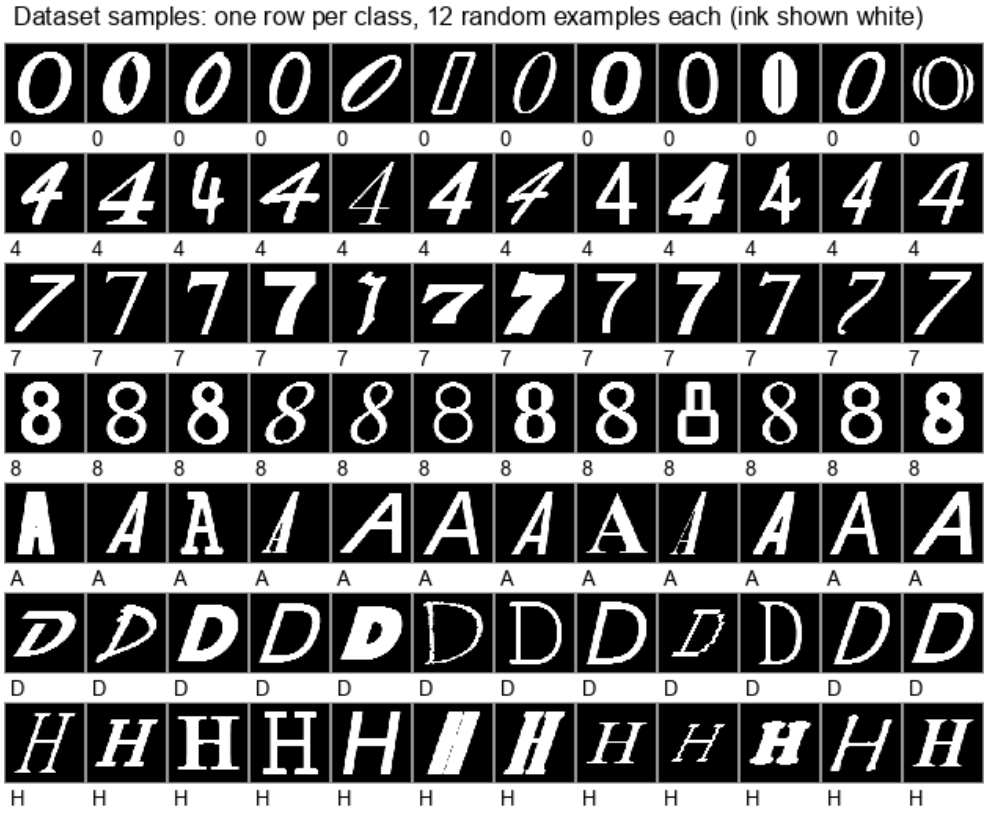


Figure 20: Examples of the seven character categories in the dataset.

Dataset Partition The dataset is divided into training, validation, and test subsets. The test set contains 25% of the images, and the remaining samples are used for model development.

Table 7: Dataset partition for character classification.

Subset	Samples	Percentage
Training	4620	64.96%
Validation	714	10.04%
Test	1778	25.00%
Total	7112	100.00%

Each category contributes 660 training, 102 validation, and 254 test images. The same partition is used for both classifiers.

Duplicate-aware Partitioning Exact duplicate images are identified using SHA-256 hashes calculated from decoded grayscale pixel arrays. Images with identical pixel content are assigned to the same subset.

The audit identifies 651 exact-duplicate groups, with no duplicate groups shared across the final training, validation, and test partitions. This prevents exact-image leakage, although visually similar images with different pixel values may remain.

7.1.3 Image Preprocessing

The original images are converted to 32×32 grayscale inputs using foreground cropping, square padding, and resizing. Cropping reduces unnecessary background regions, while square padding preserves the character aspect ratio.

Pixel intensities are normalized to $[0, 1]$:

$$I_{\text{norm}}(x, y) = \frac{I(x, y)}{255}. \tag{44}$$

The resulting representation contains bright character strokes on a dark background. Figure 21 compares the preprocessing methods evaluated in the experiments.

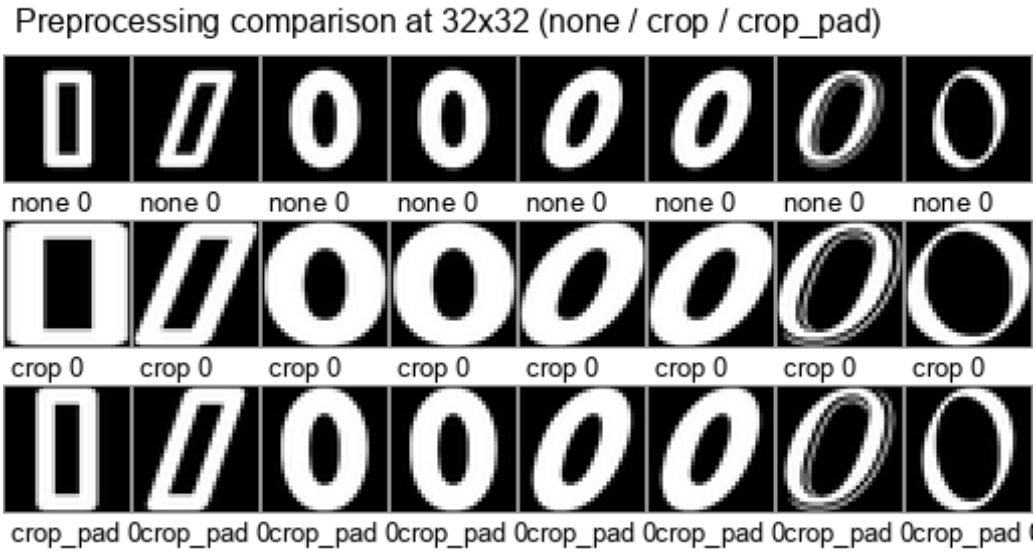


Figure 21: Comparison of character image preprocessing methods.

7.1.4 CNN Architecture

Network Structure The CNN contains two convolutional layers, two max-pooling layers, and two fully connected layers. Its architecture is illustrated in Figure 22.

The input image has dimensions $1 \times 32 \times 32$. The first convolutional layer uses 16 filters of size 3×3, followed by ReLU and 2×2 max pooling. The second layer uses 32 filters of the same size, followed by another ReLU and pooling operation.

Both convolutions use padding of one pixel, preserving spatial dimensions before pooling. After two pooling operations, the feature maps have dimensions $32 \times 8 \times 8$. Flattening produces 2,048 features, which are passed to a fully connected layer with 128 neurons, followed by ReLU, dropout, and seven output neurons.

The network contains 267,975 trainable parameters.

Convolution and Activation For input feature map X and kernel K, convolution is expressed as

$$Y_{i,j,k} = b_k + \sum_c \sum_m \sum_n K_{m,n,c,k} X_{i+m,j+n,c}. \tag{45}$$

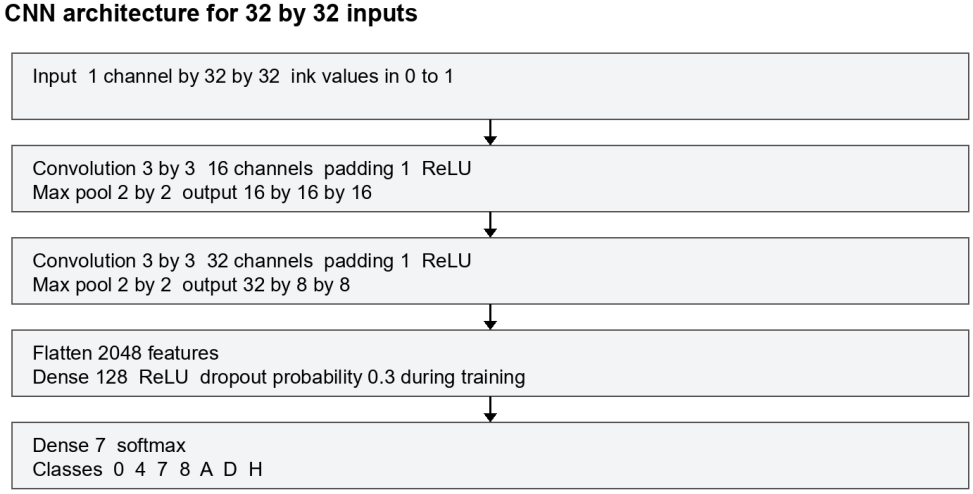


Figure 22: Architecture of the implemented CNN.

Table 8: Structure of the implemented CNN.

Layer	Configuration
Input	$1 \times 32 \times 32$
Convolution 1	16 filters, 3×3, padding 1
ReLU + Max pooling	$16 \times 16 \times 16$
Convolution 2	32 filters, 3×3, padding 1
ReLU + Max pooling	$32 \times 8 \times 8$
Flatten	2048 features
Fully connected	128 neurons, ReLU
Dropout	0.3
Output	7 neurons, softmax

The implementation uses an im2col transformation to rearrange local image patches into matrices, allowing convolution to be evaluated through matrix multiplication.

The ReLU activation function is

$$f(x) = \max(0, x). \tag{46}$$

Max pooling selects the maximum activation within each 2×2 neighborhood:

$$Y_{i,j} = \max_{(m,n)\in\Omega_{i,j}} X_{m,n}. \tag{47}$$

The final layer produces seven class scores, converted into probabilities using softmax:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^{7} \exp(z_j)}. \tag{48}$$

The predicted class corresponds to the highest output probability.

7.1.5 Training Procedure

The CNN is trained using categorical cross-entropy loss:

$$L = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{7} y_{ik} \log(p_{ik}), \tag{49}$$

where y_{ik} is the target label indicator and p_{ik} is the predicted probability.

Gradients are calculated through backpropagation, and parameters are updated using the Adam optimization algorithm [3].

Table 9: Training configuration of the selected CNN.

Parameter	Value
Input resolution	32×32
Batch size	64
Learning rate	0.003
Weight decay	0.0001
Dropout	0.3
Optimizer	Adam
Training epochs	25
Online augmentation	Disabled
Random seed	0

The network is trained for 25 epochs. Validation accuracy is measured after each epoch, and the checkpoint with the highest validation accuracy is retained.

The best validation accuracy is 99.72%, first reached at epoch 5. The recorded training procedure takes approximately 299.51 seconds.

7.1.6 Preprocessing and Hyperparameter Experiments

Seven configurations are evaluated using eight training epochs each. The experiments investigate preprocessing, online augmentation, learning rate, network size, and input resolution.

Table 10: CNN configuration experiments and validation results.

Configuration	Input	Channels	Aug.	Val. Acc.
Crop + pad	32	16/32	No	99.72%
Crop + pad	32	16/32	Yes	99.58%
Crop only	32	16/32	Yes	99.44%
Direct resize	32	16/32	Yes	99.16%
Learning rate 0.001	32	16/32	Yes	99.16%
Smaller CNN	32	8/16	Yes	99.16%
Input size 24	24	16/32	Yes	99.02%

Crop-and-pad preprocessing without online augmentation achieves the highest recorded validation accuracy of 99.72%. Enabling augmentation in the corresponding configuration produces 99.58%, indicating no improvement under the tested settings.

Direct resizing obtains a lower validation accuracy than crop-and-pad preprocessing. Preserving character proportions while reducing unnecessary background regions may contribute to this difference.

The smaller CNN contains 133,351 parameters and achieves 99.16% validation accuracy, while the 24×24 configuration contains 153,287 parameters and achieves 99.02%.

These experiments show that smaller configurations remain effective, although the selected network achieves the highest recorded validation result. The results are based on the tested configurations and do not establish universal performance rankings.

7.1.7 Training Results

Figure 23 presents the training and validation histories.

Training accuracy increases from approximately 89.74% in the first epoch to nearly 100%, while training loss decreases from 0.5193 to 0.0030.

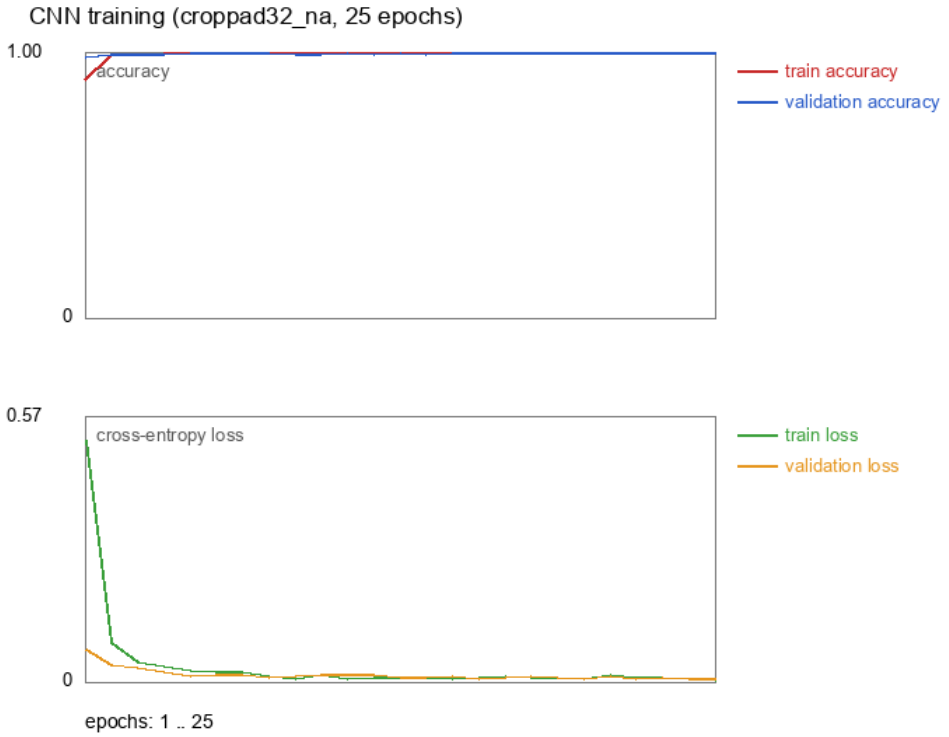


Figure 23: CNN training and validation accuracy and loss over 25 epochs.

Validation accuracy reaches 99.72% at epoch 5 and returns to this level in later epochs. The curves indicate rapid convergence, although independent test-set evaluation is required to assess generalization.

7.1.8 Test-set Evaluation

The selected CNN is evaluated on 1,778 test images. Classification accuracy is defined as

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{test}}} \times 100\%. \tag{50}$$

The CNN correctly classifies 1,764 images, giving

$$\text{Accuracy} = \frac{1764}{1778} \times 100\% \approx 99.21\%. \tag{51}$$

Table 11: CNN test-set classification performance.

Metric	Result
Test samples	1778
Correct predictions	1764
Incorrect predictions	14
Accuracy	99.21%
Macro precision	99.22%
Macro recall	99.21%
Macro F1-score	99.21%

The confusion matrix is shown in Figure 24.

The classes 0, 7, and 8 are classified correctly for all 254 samples in their respective categories. One sample of class 4 is classified as A, and one D sample as 8.

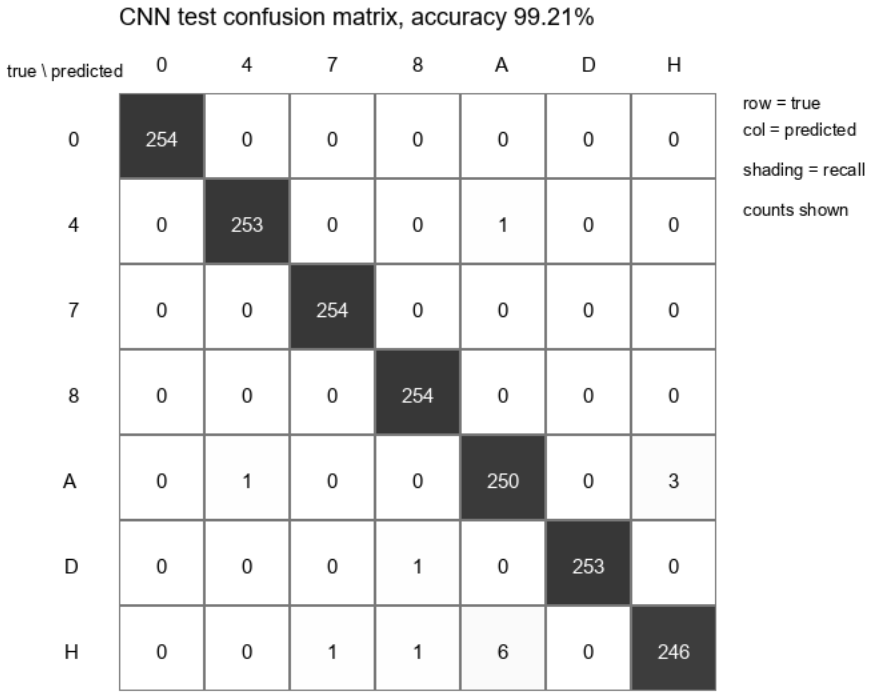


Figure 24: CNN confusion matrix on the held-out test set.

For class A, one sample is classified as 4 and three as H. Class H produces eight errors: six are classified as A, one as 7, and one as 8.

Thus, H–A confusion accounts for nine of the fourteen classification errors across both directions.

Examples of incorrectly classified images are shown in Figure 25.

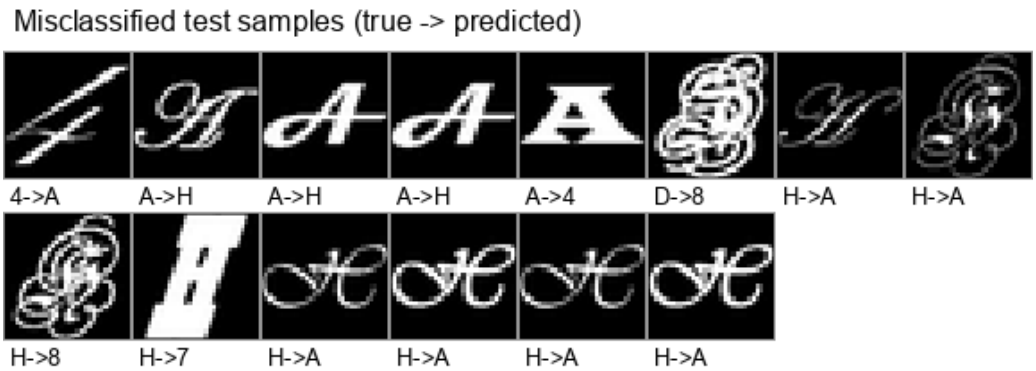


Figure 25: Examples of incorrectly classified CNN test images.

Several errors involve unusual stroke shapes or visually similar character structures. In particular, stylized H and A characters may contain similar diagonal and vertical features, making them difficult to distinguish in the reduced grayscale representation.

7.1.9 Discussion and Conclusion

The NumPy-based CNN achieves 99.21% test accuracy, demonstrating effective feature learning for the supplied seven-class dataset.

The configuration experiments indicate that preprocessing and model design influence validation performance. Crop-and-pad preprocessing produces the highest recorded validation accuracy, while the confusion matrix identifies H and A as the main source of classification errors.

Although exact duplicates are prevented from crossing dataset partitions, the evaluation does not establish performance on independent fonts or additional microchip images.

7.2 Non-CNN-based Character Classification

7.2.1 Introduction

A non-CNN classifier is developed using principal component analysis (PCA) and a support vector machine (SVM). PCA reduces feature dimensionality, while an RBF-kernel SVM performs classification in the resulting feature space.

The same dataset partition is used for both classification approaches.

7.2.2 Feature Preparation and PCA

The character images are processed using foreground cropping, square padding, and resizing to 32×32 pixels. Each image is flattened into a 1,024-dimensional feature vector and standardized before PCA.

For a centered feature matrix X, the covariance matrix is

$$C = \frac{1}{N-1} X^T X. \tag{52}$$

PCA identifies principal directions corresponding to the largest eigenvalues of the covariance matrix [4]. The transformed feature vector is

$$z = W^T(x - \mu), \tag{53}$$

where W contains the selected principal components and μ is the training-set mean.

The selected configuration retains 100 components, explaining approximately 91.61% of the standardized training-feature variance. Figure 26 shows the cumulative explained variance.

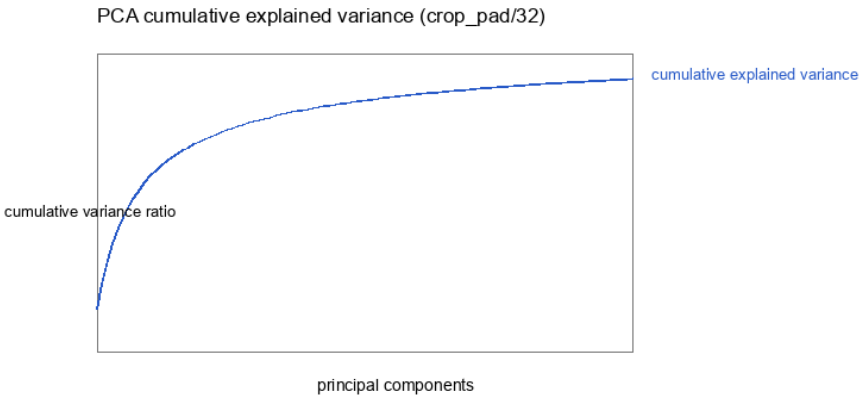


Figure 26: Cumulative explained variance of PCA components.

This reduces feature dimensionality from 1,024 to 100 before SVM classification.

7.2.3 Support Vector Machine

The support vector machine (SVM) is a margin-based classification method [5]. In this project, an RBF kernel is used:

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right). \tag{54}$$

The parameter C controls regularization, while γ determines the influence of individual samples in feature space.

The selected configuration uses

$$C = 10, \qquad \gamma = \text{scale}. \tag{55}$$

The final model contains 886 support vectors. Standardization and PCA are fitted on training data and subsequently applied to validation and test samples.

7.2.4 Preprocessing and Parameter Experiments

The preprocessing experiments are summarized in Table 12.

Table 12: PCA–SVM preprocessing experiments.

Preprocessing	Validation Accuracy
Crop + pad, 32×32	99.02%
Crop only, 32×32	99.02%
Crop + pad, 16×16	98.60%
Direct resize, 32×32	97.76%

Foreground cropping produces higher validation accuracy than direct resizing in the recorded experiments.

The influence of PCA dimensionality is summarized in Table 13.

Table 13: Influence of PCA dimensionality on validation performance.

Components	Explained Variance	Validation Accuracy
30	77.33%	98.74%
50	84.31%	98.88%
100	91.61%	99.02%
200	96.43%	99.02%

Increasing the number of components from 30 to 100 improves validation accuracy from 98.74% to 99.02%. Increasing the number further to 200 preserves more variance but does not improve accuracy.

Therefore, 100 components provide a reasonable balance between dimensionality and classification performance.

The SVM parameter experiments also show that $C = 10$ or 100, combined with $\gamma = \text{scale}$ or 0.001, achieves 99.02% validation accuracy.

Increasing γ to 0.01 reduces accuracy to approximately 92.58%, indicating sensitivity to the RBF kernel scale.

7.2.5 Test-set Evaluation

The PCA–SVM classifier is evaluated on the same 1,778 test images as the CNN.

It correctly classifies 1,762 images, achieving 99.10% accuracy.

The confusion matrix is shown in Figure 27.

The results demonstrate that PCA–SVM achieves high classification accuracy despite reducing the input feature dimension from 1,024 to 100.

7.2.6 Discussion and Conclusion

The PCA–SVM classifier achieves 99.10% test accuracy using 100 principal components and an RBF kernel.

Table 14: PCA–SVM test-set performance.

Metric	Result
Test samples	1778
Correct predictions	1762
Incorrect predictions	16
Accuracy	99.10%
Macro precision	99.11%
Macro recall	99.10%
Macro F1-score	99.10%

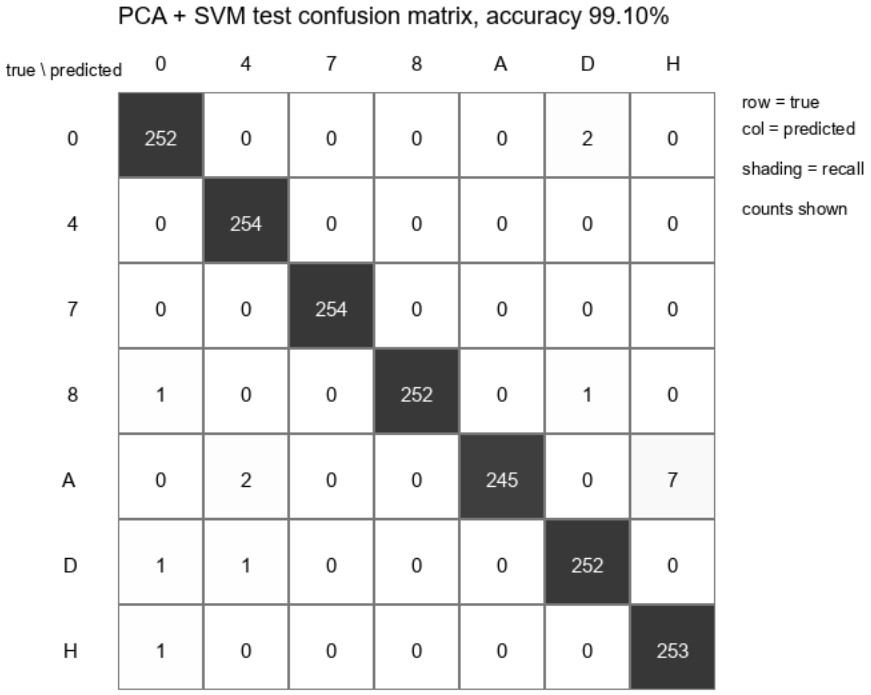


Figure 27: PCA–SVM confusion matrix on the held-out test set.

The experiments show that foreground cropping improves validation performance compared with direct resizing, while retaining more principal components does not necessarily improve classification accuracy.

The sensitivity to γ also demonstrates the importance of selecting an appropriate kernel scale.

Overall, PCA–SVM provides an effective non-CNN classification approach for the supplied dataset.

7.3 Comparison of CNN and Non-CNN Methods

7.3.1 Classification Performance

Both classifiers are evaluated using the same test partition. Their results are summarized in Table 15.

The CNN correctly classifies two more images, corresponding to an accuracy difference of approximately 0.11 percentage points.

Figure 28 compares overall classification accuracy.

Class-wise F1-scores are compared in Figure 29.

The CNN performs better for several categories, including 0 and D, while PCA–SVM achieves

Table 15: Comparison of CNN and PCA–SVM classification performance.

Metric	CNN	PCA–SVM
Accuracy	99.21%	99.10%
Macro precision	99.22%	99.11%
Macro recall	99.21%	99.10%
Macro F1-score	99.21%	99.10%
Incorrect predictions	14	16

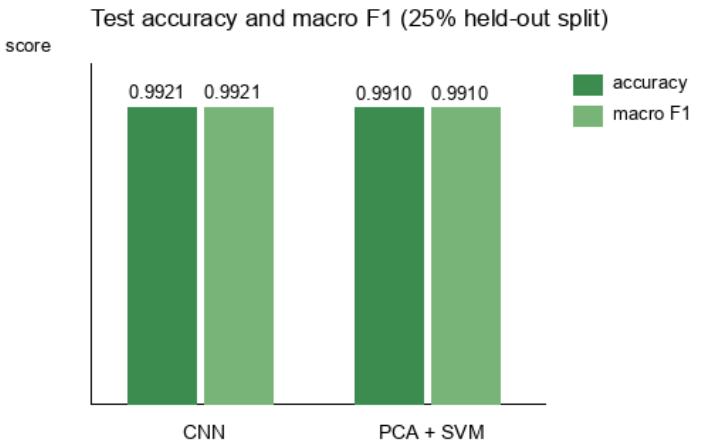


Figure 28: Comparison of CNN and PCA–SVM test accuracy.

slightly higher F1-scores for some other categories, including A and H. Neither method is uniformly superior across all classes.

7.3.2 Computational Efficiency

The recorded training and inference measurements are summarized in Table 16.

The training-time comparison is shown in Figure 30.

PCA–SVM requires substantially less recorded fitting time and achieves lower inference latency, particularly for batched predictions.

However, the timing measurements represent different scopes. CNN training time includes 25 epochs, validation, and checkpoint selection, whereas the SVM fitting time excludes preprocessing and hyperparameter search. Therefore, these values should not be interpreted as complete end-to-end development times.

The saved model sizes are relatively similar, at approximately 1.07 MB for CNN and 1.20 MB for PCA–SVM.

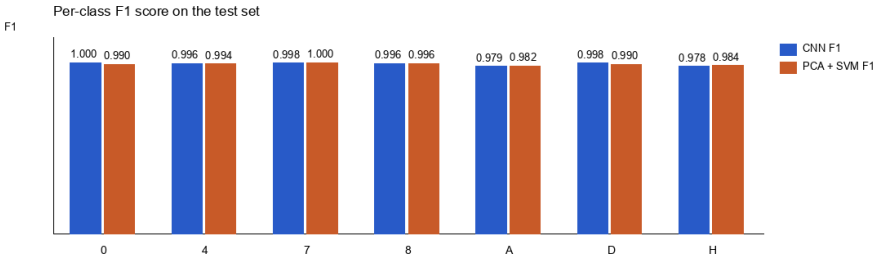


Figure 29: Per-class F1-score comparison of the two classifiers.

Table 16: Computational comparison of the classifiers.

Measurement	CNN	PCA–SVM
Training time (s)	299.51	0.96
Single-image inference (ms)	0.970	0.780
Batched inference (ms/image)	0.779	0.165
Saved model size (MB)	1.07	1.20

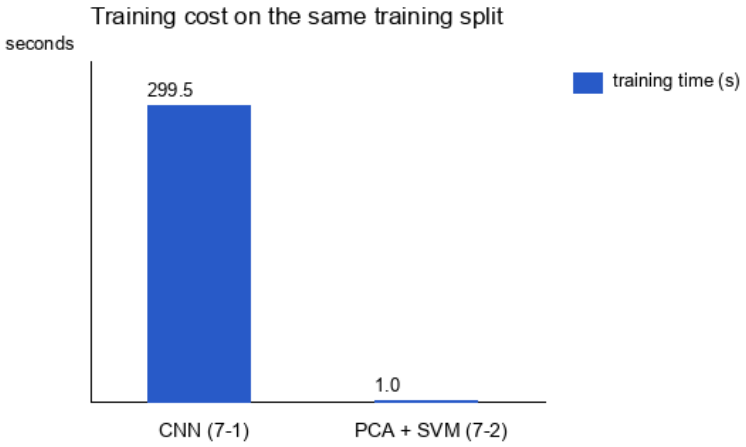


Figure 30: Recorded training times of CNN and PCA–SVM.

7.3.3 Recognition of the Microchip Character Sequence

Both classifiers are applied to the ten character regions extracted in Task 6.

Table 17: Recognition results for the target microchip sequence.

Method	Predicted Sequence	Correct
Ground truth	`HD44780A00`	–
CNN	`HD44780A00`	10/10
PCA–SVM	`H044780A00`	9/10

The CNN correctly recognizes all ten characters. PCA–SVM incorrectly classifies the second character D as 0, while the remaining nine characters are recognized correctly.

The similarity between D and 0 in certain character representations may contribute to this error. The result also shows that similar overall test accuracy does not necessarily produce identical predictions on a specific image.

7.3.4 Discussion

Both classifiers achieve approximately 99% test accuracy, but differ in computational cost and individual prediction behavior.

The CNN learns spatial features through convolutional layers and achieves slightly higher test accuracy, as well as complete recognition of the target sequence. PCA–SVM constructs lower-dimensional features and provides faster recorded fitting and inference.

The target-image experiment contains only one sequence of ten characters. Therefore, it does not establish a general performance advantage across different microchip images.

7.3.5 Conclusion

The CNN achieves 99.21% test accuracy and correctly recognizes all ten target characters, while PCA–SVM achieves 99.10% test accuracy and recognizes nine.

The results demonstrate that both methods are effective for the provided dataset, with different trade-offs between classification performance and computational efficiency.

Overall Discussion and Conclusion

Overall Performance of the Image Processing Pipeline

The image processing pipeline successfully extracts the target character sequence `HD44780A00` from the provided microchip image. The experiments demonstrate that reliable character extraction requires several complementary operations rather than relying on a single enhancement or filtering method.

The contrast enhancement and averaging-filter experiments show that improving global contrast or increasing smoothing strength does not necessarily produce better character separation. Although automatic contrast stretching increases the RMS contrast by approximately 21.34%, the textured background remains visible. Similarly, larger averaging filters suppress local intensity fluctuations but also blur character boundaries. These observations emphasize the importance of preserving character details during preprocessing.

The subsequent extraction procedure therefore uses a predefined ROI followed by multi-scale Gaussian smoothing, local background subtraction, and thresholding. The ROI excludes unrelated image regions, while background subtraction reduces slowly varying intensity variations. The threshold sensitivity experiment indicates that $T = 10$ provides a reasonable balance between background suppression and preservation of character strokes for the given image.

Connected-component filtering further reduces isolated foreground regions while retaining most character pixels. However, the remaining seven connected components do not directly correspond to the ten target characters because some neighboring strokes are connected. By combining character width estimation with vertical foreground projection, the segmentation procedure identifies three splitting positions and produces ten individual character regions.

The results demonstrate that the effectiveness of the complete pipeline depends on the interaction between preprocessing, binarization, and segmentation. Each stage reduces a particular source of interference while preserving information required by the following stage.

Comparison of Character Classification Methods

The CNN and PCA–SVM classifiers are evaluated using the same held-out test partition containing 1,778 character images. The CNN achieves 99.21% test accuracy, while PCA–SVM achieves 99.10%. The difference corresponds to only two test samples, indicating that both methods perform effectively on the supplied dataset.

The CNN learns spatial features through convolutional operations, whereas PCA–SVM reduces the input feature dimension before performing kernel-based classification. The CNN achieves slightly higher overall accuracy, but PCA–SVM requires substantially less recorded fitting time and provides lower batched inference latency.

When applied to the ten segmented microchip characters, the CNN correctly recognizes the complete sequence `HD44780A00`. PCA–SVM recognizes nine characters correctly, confusing D with 0. This result illustrates that similar test-set accuracies do not necessarily lead to identical predictions on individual character images.

The comparison also highlights the importance of considering both recognition performance and computational cost. The CNN provides a small accuracy advantage in the present experiment, while PCA–SVM offers an efficient alternative with comparable test-set performance. However, the

recorded training times represent different computational scopes, and the recognition experiment involves only one microchip image.

Limitations and Lessons Learned

Several limitations remain in the implemented recognition system.

First, the ROI is defined using fixed coordinates. Although this provides consistent extraction for the supplied image, changes in character position, scale, or orientation may require the ROI to be adjusted.

Second, the binarization and segmentation procedures depend on parameters such as Gaussian smoothing scales, the threshold, component filtering criteria, and estimated character width. The threshold sensitivity experiment demonstrates that stronger background suppression can also remove useful character details. Therefore, parameter selection must consider the preservation of character structures rather than only the reduction of foreground noise.

Third, projection-based segmentation assumes approximately horizontal character alignment and comparable character widths. Irregular spacing or extensive overlap may reduce the reliability of the estimated splitting positions.

Finally, the classification experiments use one fixed dataset partition. Although exact duplicate images are prevented from crossing partition boundaries, the results do not establish performance on independent fonts or different microchip photographs. Additional images and repeated evaluations would be necessary to assess generalization more comprehensively.

An important lesson from the project is that high classification accuracy alone is insufficient to guarantee reliable end-to-end recognition. The quality of ROI extraction, binarization, segmentation, and input normalization can influence the final prediction. Similarly, a more complex classification model does not necessarily provide a large improvement over a simpler approach for every dataset.

Future improvements could include automatic character localization, evaluation across multiple microchip images, and more systematic testing of segmentation parameters and classification robustness.

Overall Conclusion

This project develops a complete microchip character recognition procedure combining traditional image processing with CNN-based and non-CNN-based classification.

The image processing pipeline successfully separates the target sequence into ten character regions. The NumPy-based CNN achieves 99.21% test accuracy and correctly recognizes all ten target characters, while PCA–SVM achieves 99.10% test accuracy and correctly recognizes nine.

The experiments demonstrate that both classification approaches are effective for the provided dataset, with different trade-offs between accuracy and computational efficiency. More importantly, the results show that reliable character recognition depends on the combined performance of preprocessing, segmentation, and classification rather than any individual stage alone.

Implementation and Reproduction Guide

Software Environment and Project Structure

The project is implemented in Python. NumPy [6] is used for numerical computation and the implementation of the CNN, while Pillow [8] provides image reading, filtering, and other image processing operations. The non-CNN classification method uses scikit-learn [7] for PCA and SVM. The CNN, including convolution, pooling, backpropagation, and parameter optimization, is implemented using NumPy.

The implementation supports Python 3.10 or later. The recorded test environment uses Python 3.14, NumPy 2.5.3, Pillow 12.3.0, scikit-learn 1.9.1, and SciPy 1.18.1. Dependencies are listed in `requirements.txt`, with tested versions in `requirements-tested.txt`.

The main project files are summarized in Table 18.

Table 18: Main implementation files and functions.

File	Function
`segment_characters.py`	Image processing and segmentation (Tasks 1–6)
`run_task7.py`	Main execution script for Task 7
`task7/data.py`	Dataset loading, splitting, and preprocessing
`task7/cnn.py`	NumPy CNN implementation
`task7/train_cnn.py`	CNN training and configuration experiments
`task7/baseline.py`	PCA–SVM classification
`task7/compare.py`	Model performance comparison
`task7/recognize.py`	Recognition of segmented microchip characters

The dataset is supplied as `dataset_2026_updated.zip`. Generated segmentation results are stored in `output/segmentation/`, while classification results and models are stored in `output/task7/`.

Environment Setup

The following commands are intended for Windows PowerShell and should be executed from the project root directory.

Step 1: Create the Environment Create a virtual environment:

```
python -m venv .venv
```

Install the required dependencies:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

For reproduction with the recorded dependency versions, `requirements-tested.txt` may be used instead.

Step 2: Extract the Dataset Extract the dataset archive using PowerShell:

```
Expand-Archive -Path dataset_2026_updated.zip -DestinationPath dataset_2026_updated -Force
```

The extracted dataset should be located at

```
dataset_2026_updated/dataset_2026/
```

and contain the seven character categories and `Test_Image_2026.bmp`.

Execution of Tasks 1–6

Run the image processing and segmentation pipeline:

```
.\.venv\Scripts\python.exe segment_characters.py
```

The program performs contrast enhancement, spatial filtering, ROI extraction, Gaussian smoothing, background subtraction, binarization, contour detection, and character segmentation.

The default parameters are an ROI of $(10, 185, 980, 350)$, Gaussian smoothing scales of 3 and 25, threshold $T = 10$, and minimum component area of 100 pixels.

The main outputs are saved in `output/segmentation/`, including intermediate images, annotated character boxes, the vertical projection, and `segmentation.json`. Individual grayscale, binary, and normalized character images are saved in the `characters/` subdirectory.

Execution of Task 7

Run the complete classification pipeline:

```
.\.venv\Scripts\python.exe run_task7.py
```

This executes the CNN and PCA–SVM experiments, compares the classifiers, and recognizes the segmented microchip characters.

Alternatively, individual modules can be executed from the project root using the virtual environment's Python interpreter.

Table 19: Individual Task 7 execution commands.

Module command	Purpose
`python -m task7.train_cnn`	CNN training and configuration experiments
`python -m task7.baseline`	PCA–SVM training and evaluation
`python -m task7.compare`	Comparison of classification results
`python -m task7.recognize`	Recognition of segmented characters

The shared dataset partition is recorded in `output/task7/split.json`. Model files, evaluation metrics, and result figures are saved under `output/task7/`.

Verification and Expected Results

For a quick execution check, run:

```
.\.venv\Scripts\python.exe run_task7.py --quick
```

This uses reduced training settings and saves results separately in `output/task7_smoke/`, without overwriting the full-experiment outputs.

The project also provides the following verification modules:

```
python -m task7.audit
python -m task7.verify_results
python -m task7.segmentation_study
```

These commands check dataset partitioning, saved model results, and segmentation parameter sensitivity, respectively.

The expected full-experiment outputs are ten segmented character regions, a CNN test accuracy of 99.21%, and a PCA–SVM test accuracy of 99.10%.

For the target image, the recorded predictions are `HD44780A00` for CNN and `H044780A00` for PCA–SVM. The reported values correspond to the saved experimental configuration and may vary if the environment or training settings are changed.

AI Declaration

We used OpenAI Codex to assist with code development and verification, including mathematical and execution checks, and ChatGPT to assist with polishing the report text and code comments. All project decisions, experimental analyses, and conclusions were reviewed by the group. We take full responsibility for the accuracy, originality, and quality of the submitted work.

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi: 10.1109/5.726791.

[3] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015. Available: `https://arxiv.org/abs/1412.6980`.

[4] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. Springer, 2002.

[5] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995. doi: 10.1007/BF00994018.

[6] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] Pillow Contributors, “Pillow (PIL Fork) documentation.” [Online]. Available: `https://pillow.readthedocs.io/`.